

Project1实验报告

Baseline

基线模型不做任何改动，训练后结果在官网评测分数为 **0.8092**：

日期: 2025-10-14 19:28:02

分数: 0.8092

本方案在baseline上进行多种改进，运行环境为 **Ubuntu22.04-CUDA12.1.0-py311-torch2.3.1**

优化1 数据增强

先进行数据增强，考虑对数据进行一定程度上的偏移，如平移、缩放、旋转来提供多变的建筑物图形，模拟现实里无人机航拍时因无人机位移产生的同一建筑物的不同视角

```
A.ShiftScaleRotate(  
    shift_limit=0.05, # 5%的平移  
    scale_limit=0.1, # 10%的缩放  
    rotate_limit=15, # 15°的旋转  
    p=0.5, # 50%概率应用  
)
```

再进行颜色增强，就像因时间差异带来的光照条件不同导致拍摄出来的建筑物色调不一致

```
A.OneOf(  
    [  
        A.RandomContrast(),  
        A.RandomGamma(),  
        A.RandomBrightness(),  
    ],  
    p=0.4,  
)
```

不调整训练参数，此时得到的效果是

日期: 2025-10-15 17:57:33

分数: 0.8146

优化2 模型替换

原本是使用了DeepLabV3+作为模型替换，但最终结果只提升到了0.8148，提升非常小



日期: 2025-10-16 20:15:09

分数: 0.8148

于是参考论坛中第一名解决方案

通过调用smp包，将提供的FCN模型替换为Unet模型，骨干网络替换为efficientb4



```
def get_model():
    ## 初始化模型
    model = torchvision.models.segmentation.fcn_resnet50(weights=None)

    ## 修改输出层为1个通道（二分类）
    model.classifier[4] = nn.Conv2d(512, 1, kernel_size=(1, 1), stride=(1, 1))

    # 替换为Unet
    model = smp.Unet(
        encoder_name="efficientnet-b4", # choose encoder, e.g. mobilenet_v2 or efficientnet-b7
        encoder_weights='imagenet', # use `imagenet` pretrained weights for encoder initialization
        in_channels=3, # model input channels (1 for grayscale images, 3 for RGB, etc.)
        classes=1, # model output channels (number of classes in your dataset)
    )
    return model.to(DEVICE)
```

由于模型替换后训练参数不变会导致魔搭给的GPU内存溢出



```
torch.cuda.OutOfMemoryError: CUDA out of memory. Tried to allocate 256.00 MiB. GPU
```

我将参数调整，提高轮数，降低每轮使用的内存大小



```
# 增多轮数至20
EPOCHES = 20
# 从64下调为24
BATCH_SIZE = 24
# 图像尺寸不变，现在内存使用会降低到原来的约三分之一
IMAGE_SIZE = 256
```

现在的效果是：



日期: 2025-10-17 00:40:05

分数: 0.8375

优化3 TTA预测&参数微调

再引入TTA预测增强测试数据集



```
def predict_with_tta(model, img_path, tta_transform, threshold=0.5):
    """使用TTA进行预测"""
    # 读取图像
    img = cv2.imread(img_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```

original_height, original_width = img.shape[:2]

# 存储所有预测结果
all_predictions = []

# 对每个变换进行预测
for i in range(len(tta_transform)):
    # 应用变换
    transformed_img = tta_transform.apply_transform(img, i)

    # 转换为tensor
    tensor_img = TEST_TRFM(transformed_img).unsqueeze(0).to(DEVICE)

    # 模型预测
    with torch.no_grad():
        pred = model(tensor_img)[0][0].sigmoid().cpu().numpy()

    # 逆变换
    pred = tta_transform.reverse_transform(pred, i)

    # 调整到原始尺寸
    pred = cv2.resize(pred, (original_width, original_height))

    all_predictions.append(pred)

# 平均所有预测
final_pred = np.mean(all_predictions, axis=0)

# 二值化
final_mask = (final_pred > threshold).astype(np.uint8)

return final_mask

```

并且对参数进行微调，此时得到的成绩为：

日期: 2025-10-17 13:45:28

分数: 0.8678

即 **0.8678** 为最终成绩